on the CodeContests dataset (Li et al., 2022). We compare the 4-token prediction model with the next-token prediction baseline, and include a setting where the 4-token prediction model is stripped off its additional prediction heads and finetuned using the classical next-token prediction target. According to the results in Figure 4, both ways of finetuning the 4-token prediction model outperform the next-token prediction model on pass@k across k. This means the models are both better at understanding and solving the task and at generating diverse answers. Note that CodeContests is the most challenging coding benchmark we evaluate in this study. Next-token prediction finetuning on top of 4-token prediction pretraining appears to be the best method overall, in line with the classical paradigm of pretraining with auxiliary tasks followed by task-specific finetuning. Please refer to Appendix F for details.

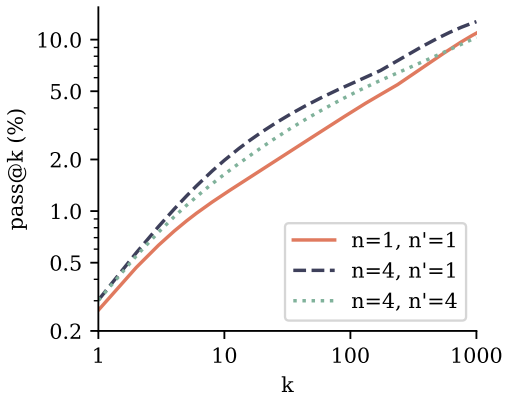


Figure 4: **Comparison of finetuning performance on CodeContests.** We finetune a 4-token prediction model on CodeContests (Li et al., 2022) (train split) using n'-token prediction as training loss with $n' = 4$ or $n' = 1$, and compare to a finetuning of the next-token prediction baseline model ($n = n' = 1$). For evaluation, we generate 1000 samples per test problem for each temperature $T \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$, and compute pass@k for each value of k and T. Shown is $k \mapsto \max_T \text{pass_at}(k, T)$, i.e. we grant access to a temperature oracle. We observe that both ways of finetuning the 4-token prediction model outperform the next-token prediction baseline. Intriguingly, using next-token prediction finetuning on top of the 4-token prediction model appears to be the best method overall.

3.7. Multi-token prediction on natural language

To evaluate multi-token prediction training on natural language, we train models of size 7B parameters on 200B tokens of natural language with a 4-token, 2-token and next-token prediction loss, respectively. In Figure 5, we evaluate the resulting checkpoints on 6 standard NLP benchmarks. On these benchmarks, the 2-future token prediction model performs on par with the next-token prediction baseline

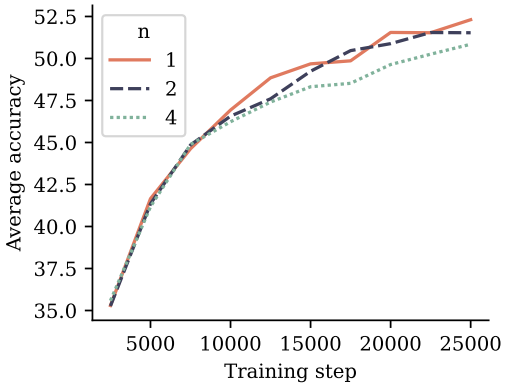


Figure 5: **Multi-token training with 7B models doesn't improve performance on choice tasks.** This figure shows the evolution of average accuracy of 6 standard NLP benchmarks. Detailed results in Appendix G for 7B models trained on 200B tokens of language data. The 2 future token model has the same performance as the baseline and the 4 future token model regresses a bit. Larger model sizes might be necessary to see improvements on these tasks.

throughout training. The 4-future token prediction model suffers a performance degradation. Detailed numbers are reported in Appendix G.

However, we do not believe that multiple-choice and likelihood-based benchmarks are suited to effectively discern *generative capabilities* of language models. In order to avoid the need for human annotations of generation quality or language model judges—which comes with its own pitfalls, as pointed out by Koo et al. (2023)—we conduct evaluations on summarization and natural language mathematics benchmarks and compare pretrained models with training sets sizes of 200B and 500B tokens and with next-token and multi-token prediction losses, respectively.

For summarization, we use eight benchmarks where ROUGE metrics (Lin, 2004) with respect to a ground-truth summary allow automatic evaluation of generated texts. We finetune each pretrained model on each benchmark's training dataset for three epochs and select the checkpoint with the highest ROUGE-L F_1 score on the validation dataset. Figure 6 shows that multi-token prediction models with both $n = 2$ and $n = 4$ improve over the next-token baseline in ROUGE-L F_1 scores for both training dataset sizes, with the performance gap shrinking with larger dataset size. All metrics can be found in Appendix H.

For natural language mathematics, we evaluate the pretrained models in 8-shot mode on the GSM8K benchmark (Cobbe et al., 2021) and measure accuracy of the final answer produced after a chain-of-thought elicited by the few-shot examples. We evaluate pass@k metrics to quantify diversity and correctness of answers like in code evaluations

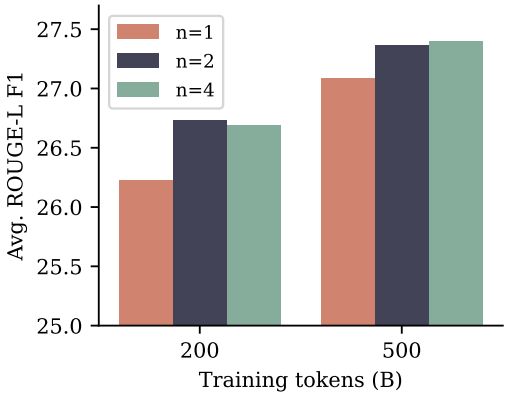


Figure 6: **Performance on abstractive text summarization.** Average ROUGE-L (longest common subsequence overlap) F_1 score for 7B models trained on 200B and 500B tokens of natural language on eight summarization benchmarks. We finetune the respective models on each task's training data separately for three epochs and select the checkpoints with highest ROUGE-L F_1 validation score. Both $n = 2$ and $n = 4$ multi-token prediction models have an advantage over next-token prediction models. Individual scores per dataset and more details can be found in Appendix H.

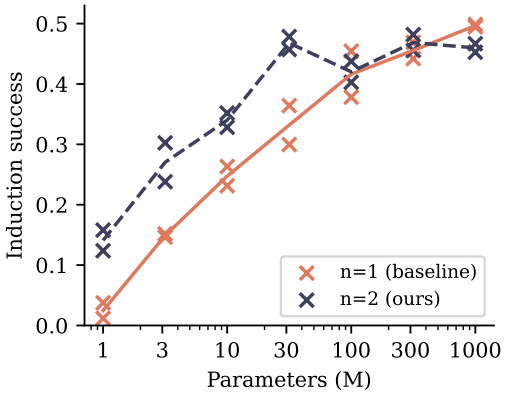


Figure 7: **Induction capability of n-token prediction models.** Shown is accuracy on the second token of two token names that have already been mentioned previously. Shown are numbers for models trained with a next-token and a 2-token prediction loss, respectively, with two independent runs each. The lines denote per-loss averages. For small model sizes, next-token prediction models learn practically no or significantly worse induction capability than 2-token prediction models, with their disadvantage disappearing at the size of 100M nonembedding parameters.

and use sampling temperatures between 0.2 and 1.4. The results are depicted in Figure S13 in Appendix I. For 200B training tokens, the $n = 2$ model clearly outperforms the next-token prediction baseline, while the pattern reverses after 500B tokens and $n = 4$ is worse throughout.

4. Ablations on synthetic data

What drives the improvements in downstream performance of multi-token prediction models on all of the tasks we have considered? By conducting toy experiments on controlled training datasets and evaluation tasks, we demonstrate that multi-token prediction leads to *qualitative changes in model capabilities and generalization behaviors*. In particular, Section 4.1 shows that for small model sizes, *induction capability*—as discussed by Olsson et al. (2022)—either only forms when using multi-token prediction as training loss, or it is vastly improved by it. Moreover, Section 4.2 shows that multi-token prediction improves generalization on an arithmetic task, even more so than tripling model size.

4.1. Induction capability

Induction describes a simple pattern of reasoning that completes partial patterns by their most recent continuation (Olsson et al., 2022). In other words, if a sentence contains "AB" and later mentions "A", induction is the prediction that the continuation is "B". We design a setup to measure induction capability in a controlled way. Training small models of sizes 1M to 1B nonembedding parameters on a dataset of children stories, we measure induction capability by means of an adapted test set: in 100 stories from the original test split, we replace the character names by randomly generated names that consist of two tokens with the tokenizer we employ. Predicting the first of these two tokens is linked to the semantics of the preceding text, while predicting the second token of each name's occurrence after it has been mentioned at least once can be seen as a pure induction task. In our experiments, we train for up to 90 epochs and perform early stopping with respect to the test metric (i.e. we allow an epoch oracle). Figure 7 reports induction capability as measured by accuracy on the names' second tokens in relation to model size for two runs with different seeds.

We find that 2-token prediction loss leads to a vastly improved formation of induction capability for models of size 30M nonembedding parameters and below, with their advantage disappearing for sizes of 100M nonembedding parameters and above.[1] We interpret this finding as follows: multi-token prediction losses help models to learn transferring information across sequence positions, which lends itself to the formation of induction heads and other in-context learning mechanisms. However, once induction capability has been formed, these *learned features* transform induction

[1]Note that a perfect score is not reachable in this benchmark as some of the tokens in the names in the evaluation dataset never appear in the training data, and in our architecture, embedding and unembedding parameters are not linked.

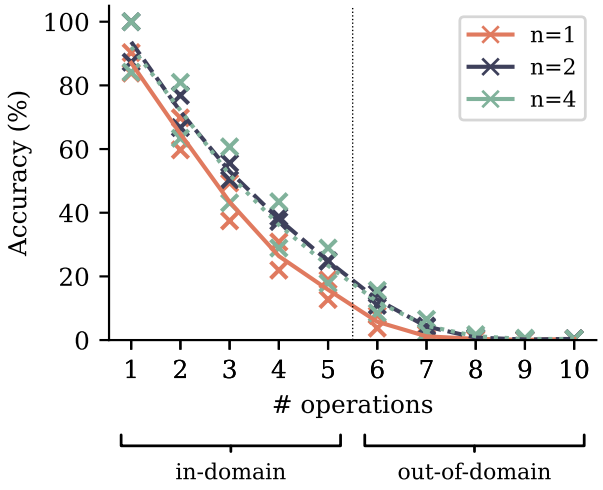


Figure 8: **Accuracy on a polynomial arithmetic task with varying number of operations per expression.** Training with multi-token prediction losses increases accuracy across task difficulties. In particular, it also significantly improves out-of-domain generalization performance, albeit at a low absolute level. Tripling the model size, on the other hand, has a considerably smaller effect than replacing next-token prediction with multi-token prediction loss (Figure S16). Shown are two independent runs per configuration with 100M parameter models.

into a task that can be solved *locally* at the current token and learned with next-token prediction alone. From this point on, multi-token prediction actually hurts on this restricted benchmark—but we surmise that there are higher forms of in-context reasoning to which it further contributes, as evidenced by the results in Section 3.1. In Figure S14, we provide evidence for this explanation: replacing the children stories dataset by a higher-quality 9:1 mix of a books dataset with the children stories, we enforce the formation of induction capability early in training by means of the dataset alone. By consequence, except for the two smallest model sizes, the advantage of multi-token prediction on the task disappears: feature learning of induction features has converted the task into a pure next-token prediction task.

4.2. Algorithmic reasoning

Algorithmic reasoning tasks allow to measure more involved forms of in-context reasoning than induction alone. We train and evaluate models on a task on polynomial arithmetic in the ring $\mathbb{F}_7[X]/(X^5)$ with unary negation, addition, multiplication and composition of polynomials as operations. The coefficients of the operands and the operators are sampled uniformly. The task is to return the coefficients of the polynomials corresponding to the resulting expressions. The number m of operations contained in the expressions is selected uniformly from the range from 1 to 5 at training time, and can be used to adjust the difficulty of both in-domain ($m \leq 5$) and out-of-domain ($m > 5$) generalization evaluations. The evaluations are conducted with greedy sampling on a fixed test set of 2000 samples per number of operations. We train models of two small sizes with 30M and 100M nonembedding parameters, respectively. This simulates the conditions of large language models trained on massive text corpora which are likewise under-parameterized and unable to memorize their entire training datasets.

Multi-token prediction improves algorithmic reasoning capabilities as measured by this task across task difficulties (Figure 8). In particular, it leads to impressive gains in out-of-distribution generalization, despite the low absolute numbers. Increasing the model size from 30M to 100M parameters, on the other hand, does not improve evaluation accuracy as much as replacing next-token prediction by multi-token prediction does (Figure S16). In Appendix K, we furthermore show that multi-token prediction models retain their advantage over next-token prediction models on this task when trained and evaluated with *pause tokens* (Goyal et al., 2023).

5. Why does it work? Some speculation

Why does multi-token prediction afford superior performance on coding evaluation benchmarks, and on small algorithmic reasoning tasks? Our intuition, developed in this section, is that multi-token prediction mitigates the distributional discrepancy between training-time teacher forcing and inference-time autoregressive generation. We support this view with an illustrative argument on the *implicit weights* multi-token prediction assigns to tokens depending on their relevance for the continuation of the text, as well as with an information-theoretic decomposition of multi-token prediction loss.

5.1. Lookahead reinforces choice points

Not all token decisions are equally important for generating useful texts from language models (Bachmann and Nagarajan, 2024; Lin et al., 2024). While some tokens allow stylistic variations that do not constrain the remainder of the text, others represent *choice points* that are linked with higher-level semantic properties of the text and may decide whether an answer is perceived as useful or *derailing*.

Multi-token prediction implicitly assigns weights to training tokens depending on how closely they are correlated with their successors. As an illustrative example, consider the sequence depicted in Figure 9 where one transition is a hard-to-predict choice point while the other transitions are considered "inconsequential". Inconsequential transitions following a choice point are likewise hard to predict in advance. By marking and counting loss terms, we find that

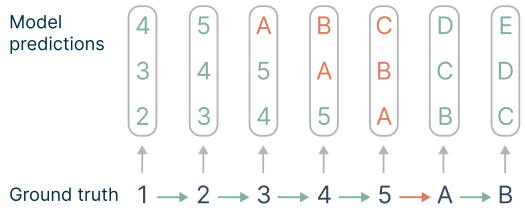


Figure 9: **Multi-token prediction loss assigns higher implicit weights to *consequential* tokens.** Shown is a sequence in which all transitions except "5 → A" are easy to predict, alongside the corresponding prediction targets in 3-token prediction. Since the consequences of the difficult transition "5 → A" are likewise hard to predict, this transition receives a higher implicit weight in the overall loss via its correlates "3 → A", ..., "5 → C".

n-token prediction associates a weight of $\frac{n(n+1)}{2}$ to choice points via their correlates, and a smaller weight of n to inconsequential points. Please refer to Appendix L.3 for more details. Generally, we believe that the quality of text generations depends on picking the right decisions at choice points, and that n-token prediction losses promote those.

5.2. Information-theoretic argument

Language models are typically trained by teacher-forcing, where the model receives the ground truth for each future token during training. However, during test time generation is unguided and autoregressive, whereby errors accumulate. Teacher-forcing, we argue, encourages models to focus on predicting well in the very short term, at the potential expense of ignoring longer-term dependencies in the overall structure of the generated sequence.

To illustrate the impact of multi-token prediction, consider the following information-theoretic argument. Here, X denotes the next future token, and Y the second-next future token. The production of both of these tokens is conditioned on some observed, input context C, that we omit from our equations for simplicity. When placed before token X, vanilla next-token prediction concerns the quantity $H(X)$, while multi-token prediction with $n = 2$ aims at $H(X) + H(Y)$. We decompose these two quantities as:

$$H(X) = H(X \mid Y) + I(X;Y),$$
$$H(X) + H(Y) = H(X \mid Y) + 2I(X;Y) + H(Y \mid X).$$

By discarding the term $H(Y \mid X)$—which appears again when predicting at the following position—we observe that 2-token prediction increases the importance of $I(X;Y)$ by a factor of 2. So, multi-token predictors are more accurate at predicting tokens X that are of relevance for the remainder of the text to come. In Appendix L.2, we give a relative version of the above equations that shows the increased weight of *relative mutual information* in a loss decomposition of 2-token prediction loss.

6. Related work

Language modeling losses Dong et al. (2019) and Tay et al. (2022) train on a mixture of denoising tasks with different attention masks (full, causal and prefix attention) to bridge the performance gap with next token pretraining on generative tasks. Tay et al. (2022) uses the span corruption objective, which replaces spans of tokens with special tokens for the encoder and the decoder then predicts the contents of those spans. Unlike UniLM, this allows full causal training with teacher forcing. Similarly, Yang et al. (2019) train on permuted sequences, while conserving the original positional embeddings, effectively training the model to predict various parts of the sequence given a mix of past and future information. This permuted language modeling is the closest task to ours since it allows predicting beyond the next token. However all of these language modeling tasks train on a small percentage of the input text: on average only 15% of the tokens are backwarded through. For Dong et al. (2019), where the masking is done in BERT style, it is hard to mask more than 15% since it destroys too much information. For Tay et al. (2022), it is technically possible to have a larger proportion but in practice, the settings used have between 15% and 25% of masked tokens. (Yang et al., 2019) also makes it possible to train on the whole sequence since it is only permuted, and no information is lost. Yet, in practice, since the completely random permutation is very hard to reconstruct, only 15% are predicted for training stability reasons.

Multi-token prediction in language modelling Qi et al. (2020) argue that multi-token prediction encourages planning, improves representations and prevents the overfitting on local patterns that can result from teacher-forced training. However, their technical approach replicates the residual stream n-fold while ours allows for compute-matched comparisons and makes the residual representations participate more directly in the auxiliary loss terms. Stern et al. (2018) and Cai et al. (2024) propose model finetunings with multi-token prediction for faster inference but do not study the effects of such a loss during pretraining. Pal et al. (2023) use probing methods to show that next-token prediction models are able to predict additional consecutive tokens to a certain extent, but less so than our models which are specifically trained for this task. Jianyu Zhang (2024) observe improvements in language modelling tasks with multi-label binary classification over the occurrence of vocabulary words in the future as an auxiliary learning task.